

RAI-818: INTELLIGENT **MOBILE ROBOTICS**

Cell Decomposition

- **Exact Cell Decomposition**
- **Approximate Cell Decomposition**

Text : Principles of Robot Motion – Theory, Algorithms and Implementations
H. Choset, K.M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, et al.

INSTRUCTOR:

DR. KUNWAR FARAZ AHMED KHAN

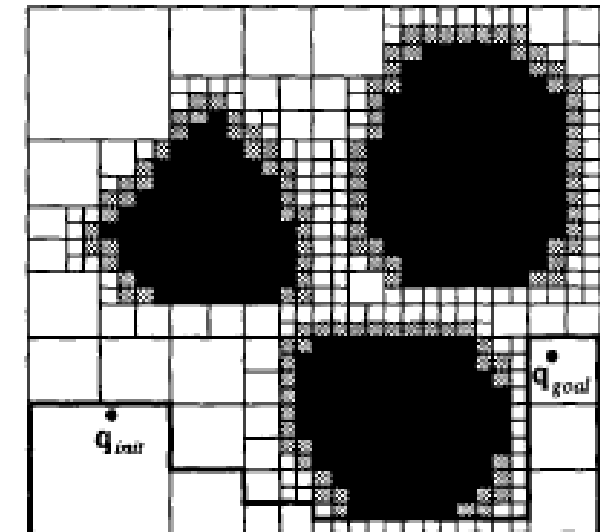
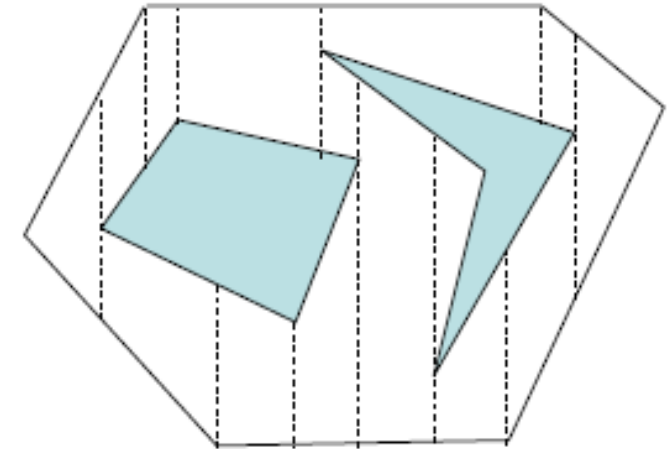
Cell Decomposition



- Cell Decomposition
- Basic Idea

The Basic Idea

- In this chapter we consider a different type of representation of a free space called *cell decomposition*
- These structures represent the *free space* by a union of *simple* regions called *cells*.
- Shared boundaries of cells often have a *physical meaning*
 - Change in the closest obstacle
 - Change in the line of sight to surrounding obstacles
- Two cells are *adjacent* if they share a common boundary



Defining futures

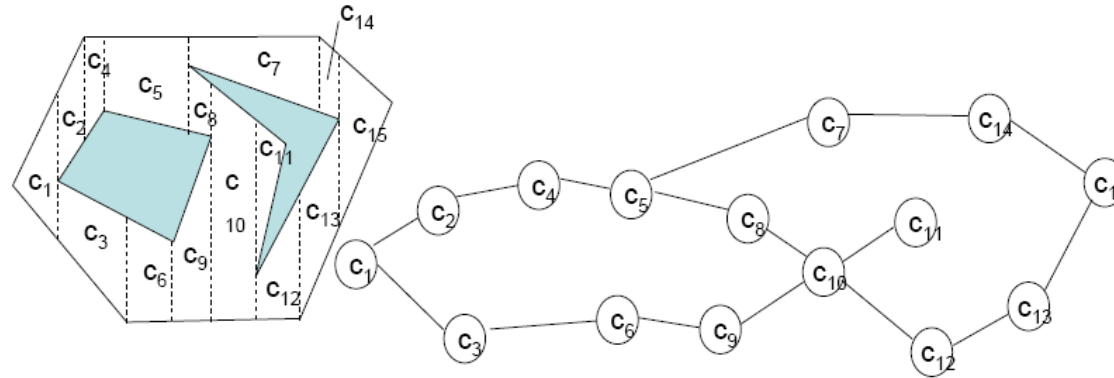
Cell Decomposition



- Cell Decomposition
- Basic Idea

The Basic Idea

- Path Planning in two steps:
 - Planner determines *cells* that contain the *start* and *goal*
 - Planner searches for a *path* within *adjacency graph*



- Coverage
 - Cell decompositions can be used to achieve *coverage*
 - *Coverage* path planner determines a path that passes an end effector (e.g., a robot, a detector) over all points in a free space
 - Since each cell has a *simple* structure, so it can be covered with simple motions (back and forth motion)
 - Once a robot has visited each *cell* the *coverage* is complete



Defining futures

Cell Decomposition



- Cell Decomposition
 - Basic Idea
 - **Types**

Types of Cell Decomposition: Exact and Approximate

- *Exact Cell Decomposition:*
 - set of cells that *exactly* covers the C_{free}
 - complicated cells with irregular boundaries
 - harder to compute
- *Approximate Cell Decomposition:*
 - set of cells that *approximately* covers C_{free}
 - simpler cells with regular boundaries
 - easier to compute



Defining futures

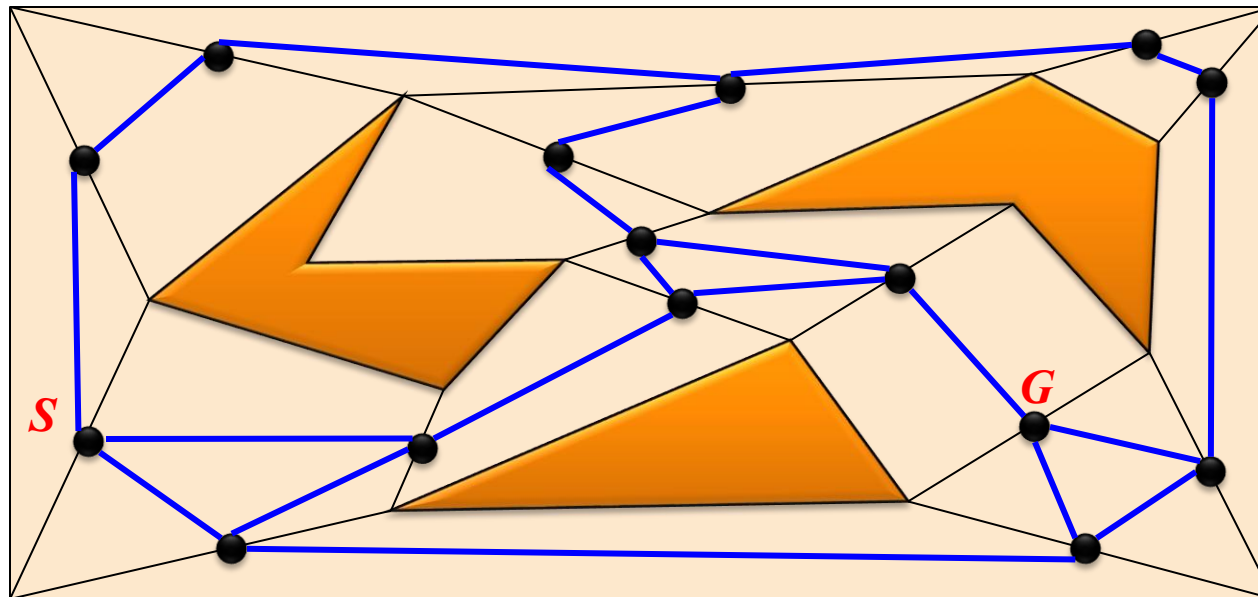
Cell Decomposition



- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Exact Decomposition

- Collection of non-overlapping cells, such that the $\text{Union}(\text{Cells}) = \text{Free Space}$
- Finite set of convex polygons that cover the space
- Midpoints of adjacent cells are crossings



Defining futures

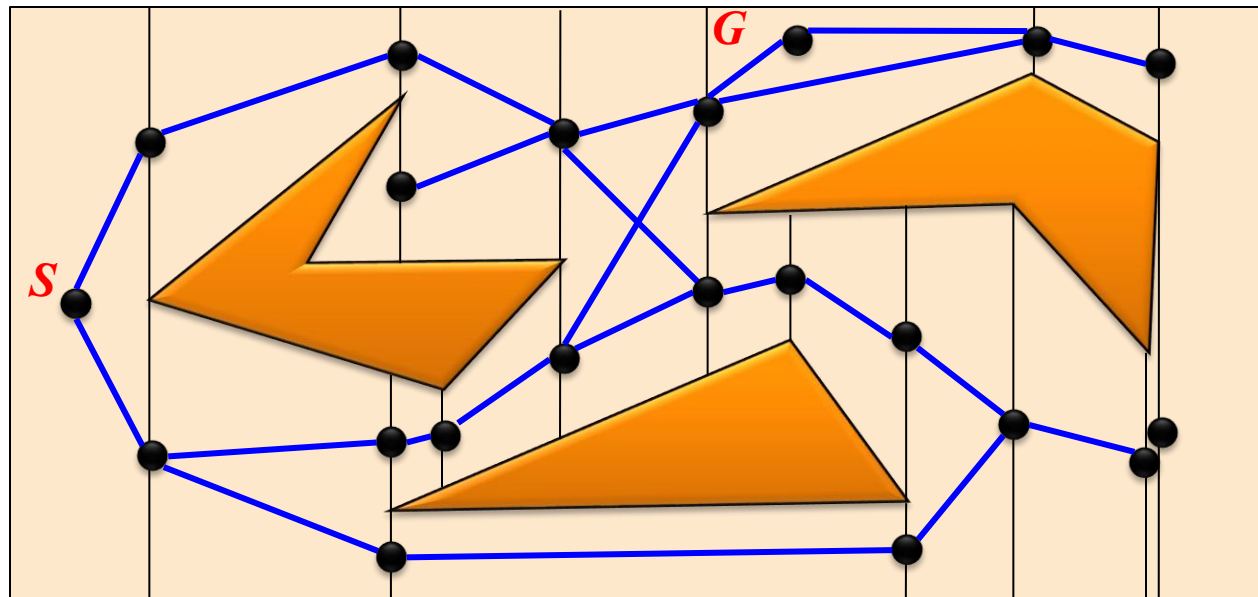
Cell Decomposition



- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

- Collection of non-overlapping cells, such that the $\text{Union}(\text{Cells}) = \text{Free Space}$
- Extend a bi-directional vertical line from each vertex until collision
- This will give you trapezoids



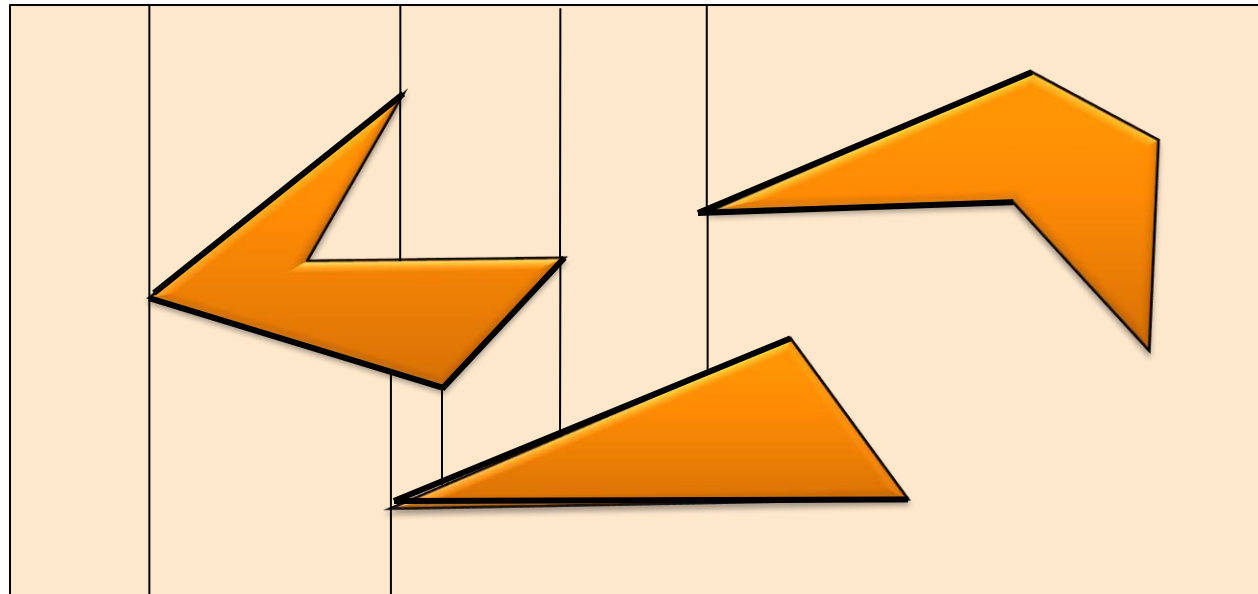
Cell Decomposition



- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

- How do we compute this efficiently?
- *Plane Sweep Algorithm*



Defining futures

Cell Decomposition



- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

- Plane sweep Algorithm
 - **Input:** Boundary representation of configuration space
 - **Output:** Trapezoidal decomposition of configuration space
 1. Sort vertices of the configuration space by x -coordinates
 2. *Sweep* a vertical line L from left to right
 3. Stop at each vertex v in the configuration space

(Note: v is incident on exactly two edges e_1 and e_2 in configuration space)



Defining futures



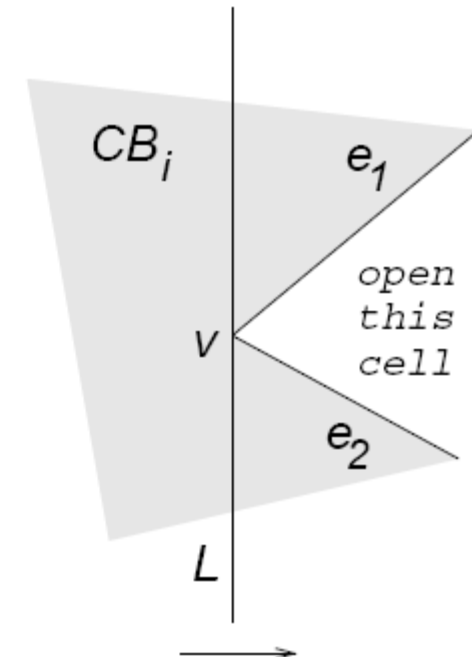
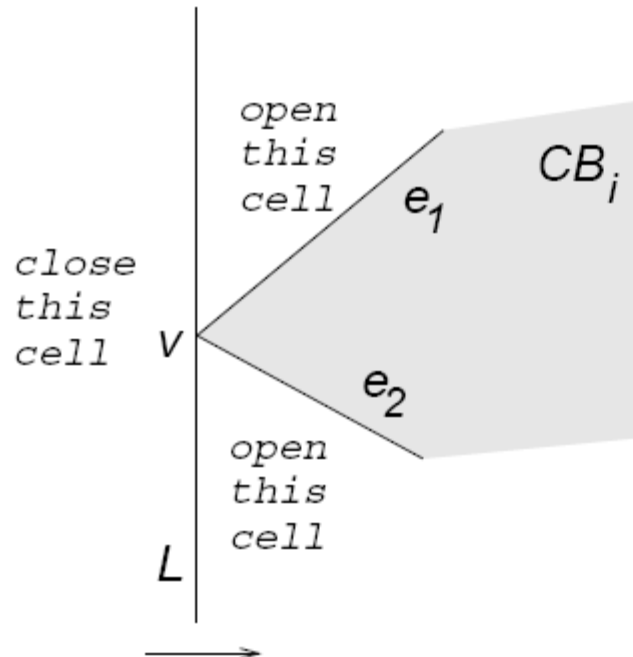
Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

- Plane sweep Algorithm
 - **Input:** Boundary representation of configuration space
 - **Output:** Trapezoidal decomposition of configuration space

Case 1: Both e_1 and e_2 lie to the right of L



Defining futures

Cell Decomposition

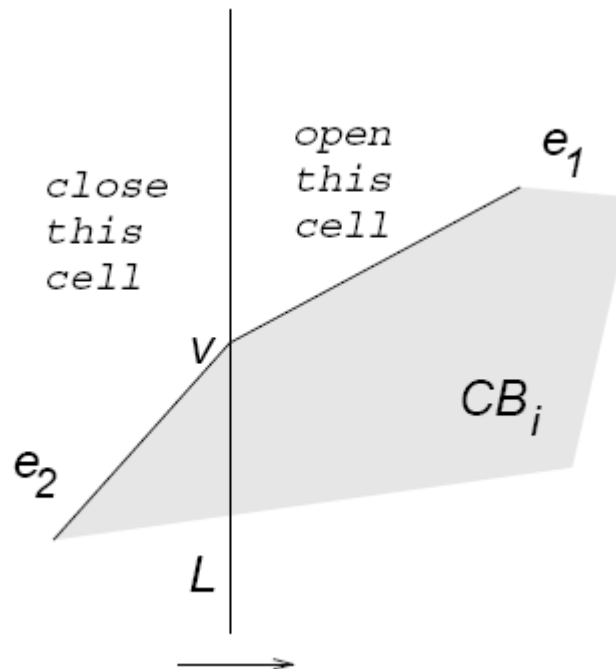


- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

- Plane sweep Algorithm
 - **Input:** Boundary representation of configuration space
 - **Output:** Trapezoidal decomposition of configuration space

Case 2: Exactly one of e_1 and e_2 lie to the right of L



Defining futures



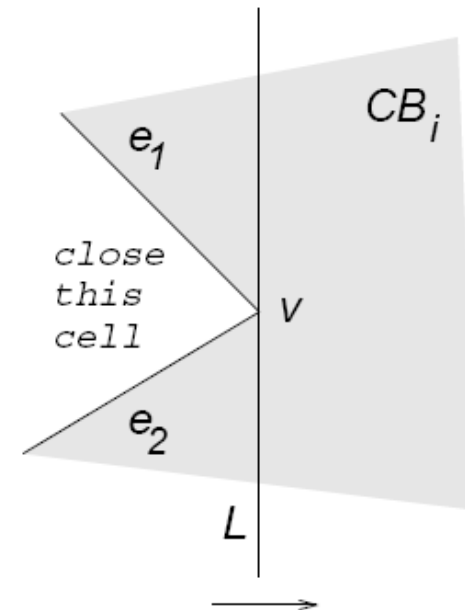
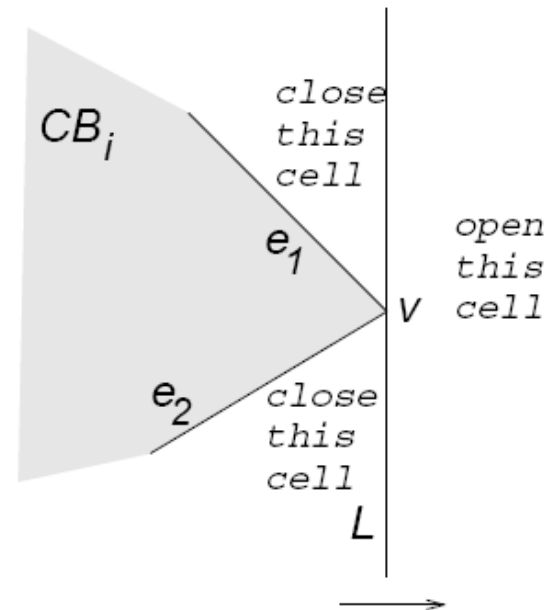
Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

- Plane sweep Algorithm
 - **Input:** Boundary representation of configuration space
 - **Output:** Trapezoidal decomposition of configuration space

Case 3: Both e_1 and e_2 lie to the left of L



Defining futures



Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

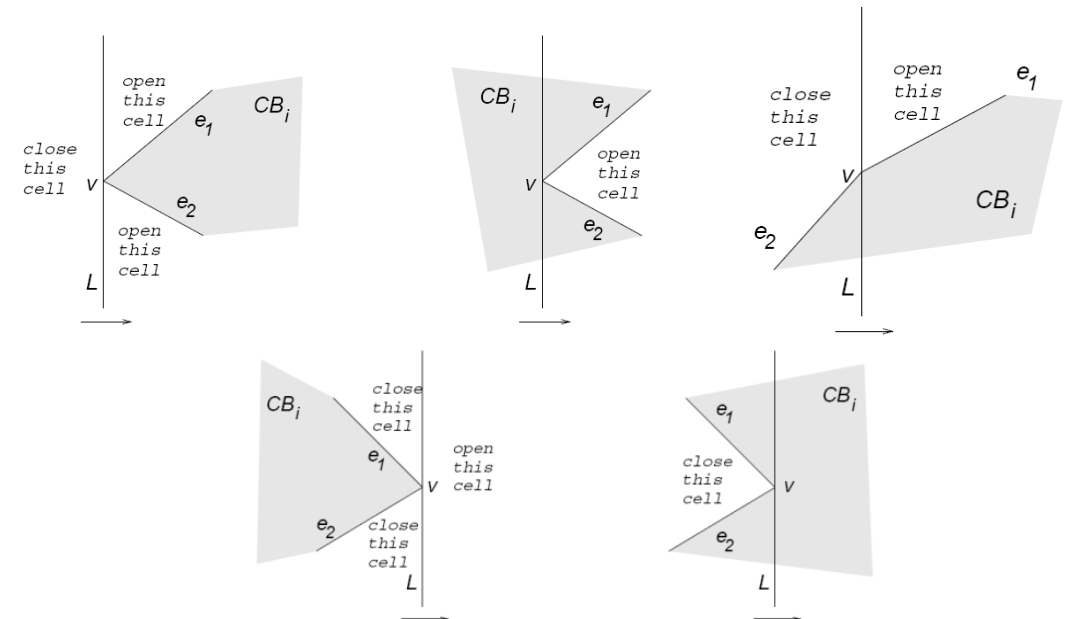
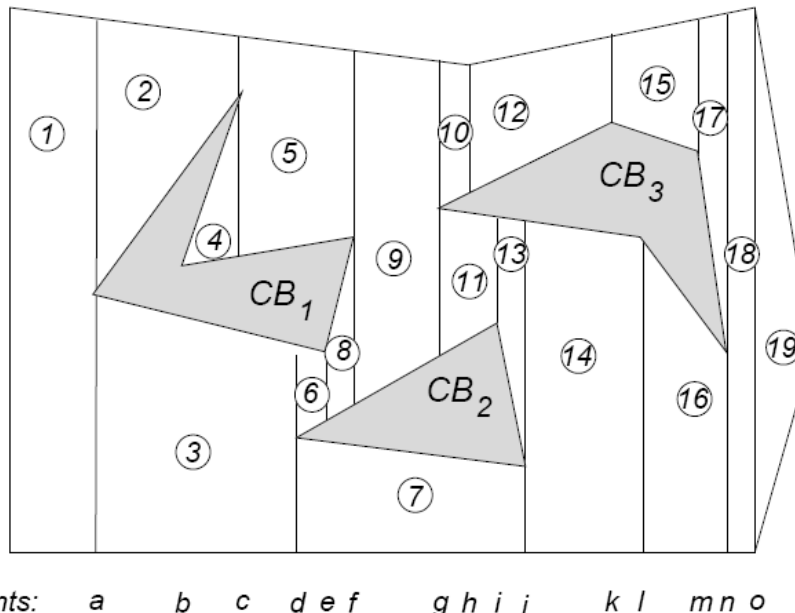
- Plane sweep Algorithm – Example

Event a (case 1(a)):

just prior to *Event a* , L intersects *cell 1*

at *Event a* we close *cell 1* and open *cells 2* and *3*

just after *Event a* , L intersects *cells 2* and *3* (in that order)



Defining futures



Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

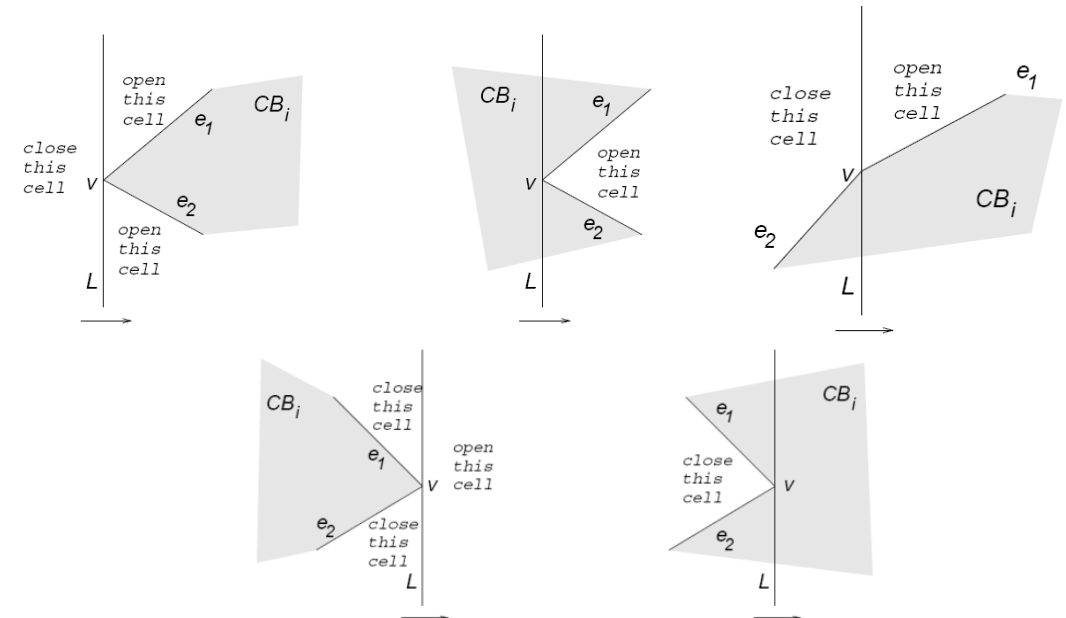
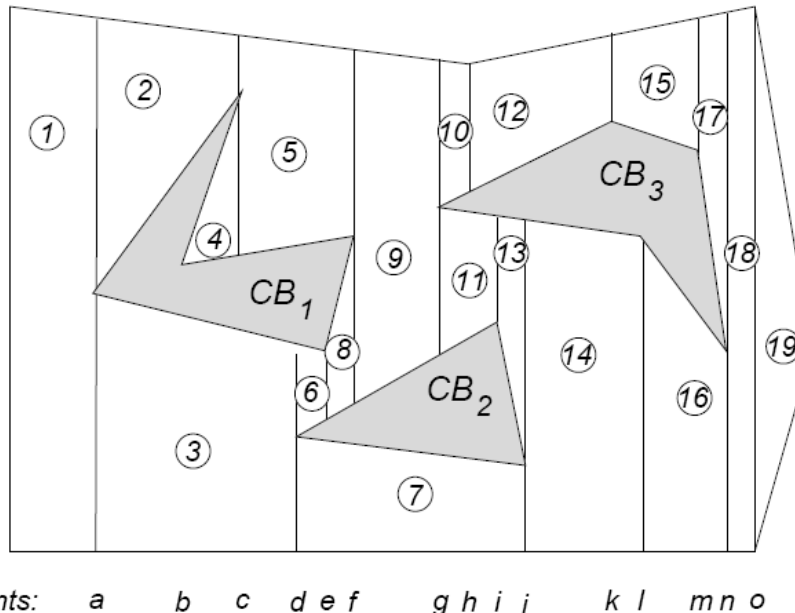
- Plane sweep Algorithm – Example

Event b (case 1(b)):

just prior to *Event b* , L intersects **cells 2** and **3** (in that order)

at *Event b* we open **cell 4**

just after *Event b* , L intersects **cells 2, 4, and 3** (in that order)



Defining futures



Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

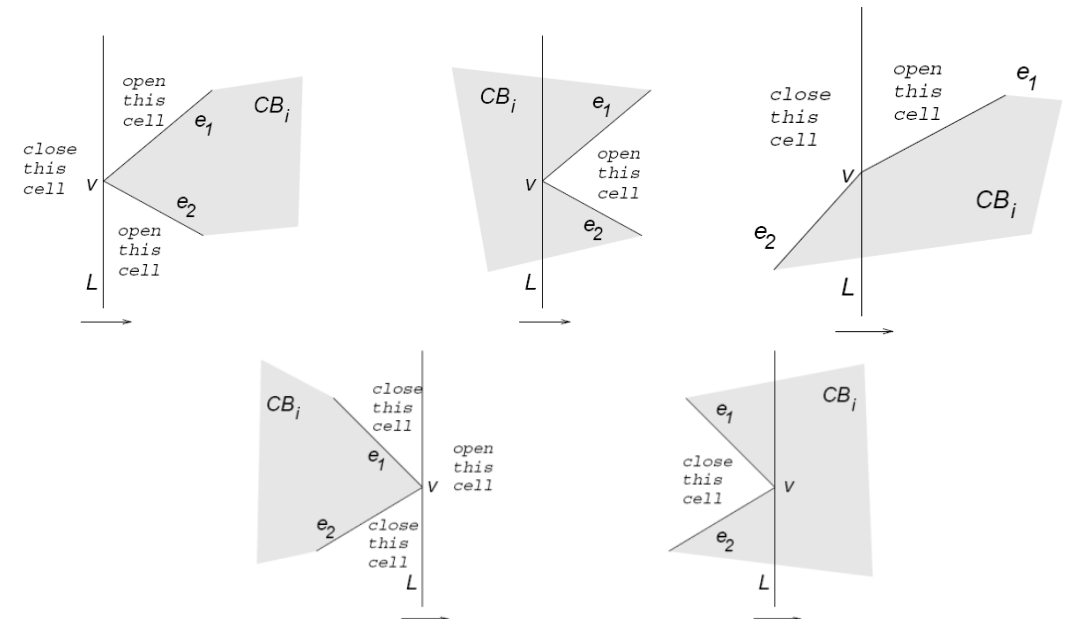
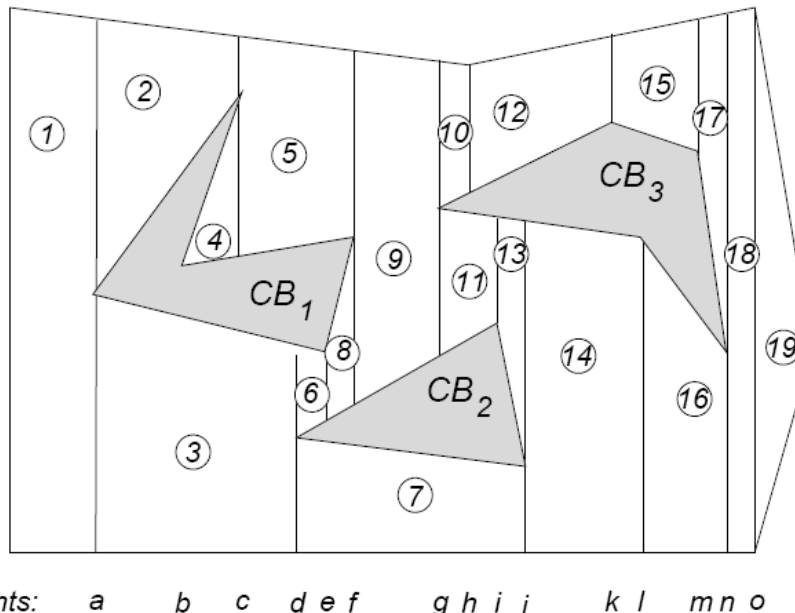
- Plane sweep Algorithm – Example

Event c (case 3(a)):

just prior to **Event c**, L intersects *cells 2, 4, and 3* (in order)

at **Event c** we close *cells 2 and 4* and open *cell 5*

just after **Event c**, L intersects *cells 5 and 3* (in that order)



Defining futures



Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

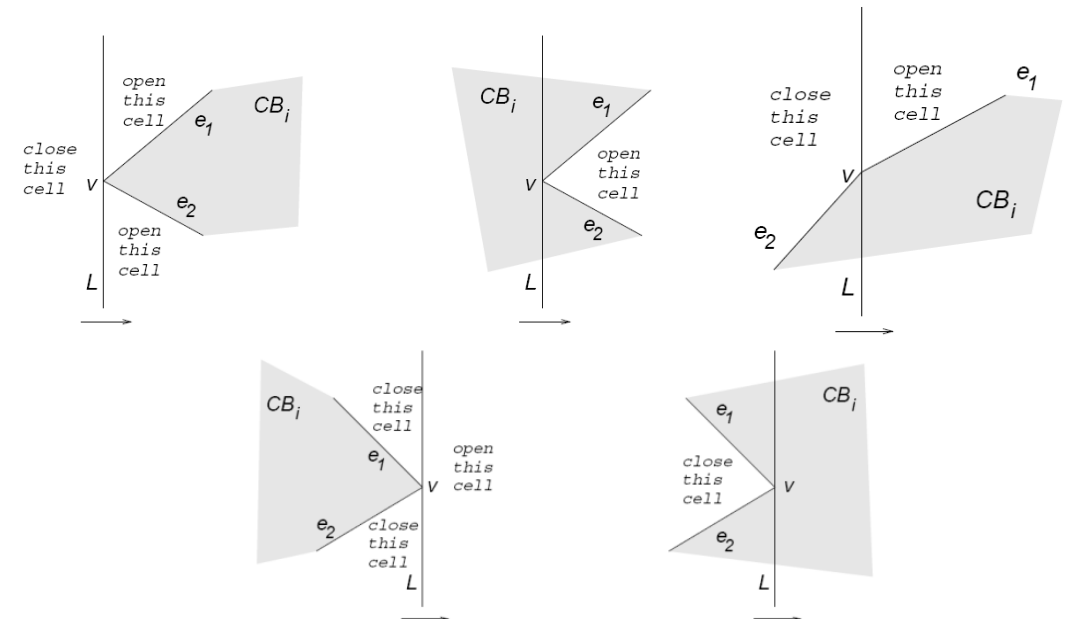
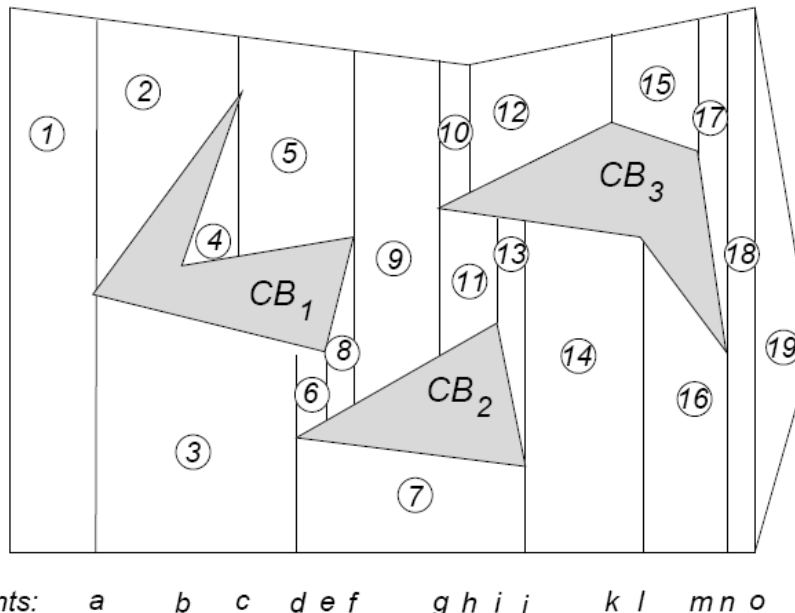
- Plane sweep Algorithm – Example

Event e (case 2):

just prior to *Event e* , L intersects *cells 5, 6, and 7* (in order)

at *Event e* we close *cell 6* and open *cell 8*

just after *Event e* , L intersects *cells 5, 8, and 7* (in order)



Defining futures

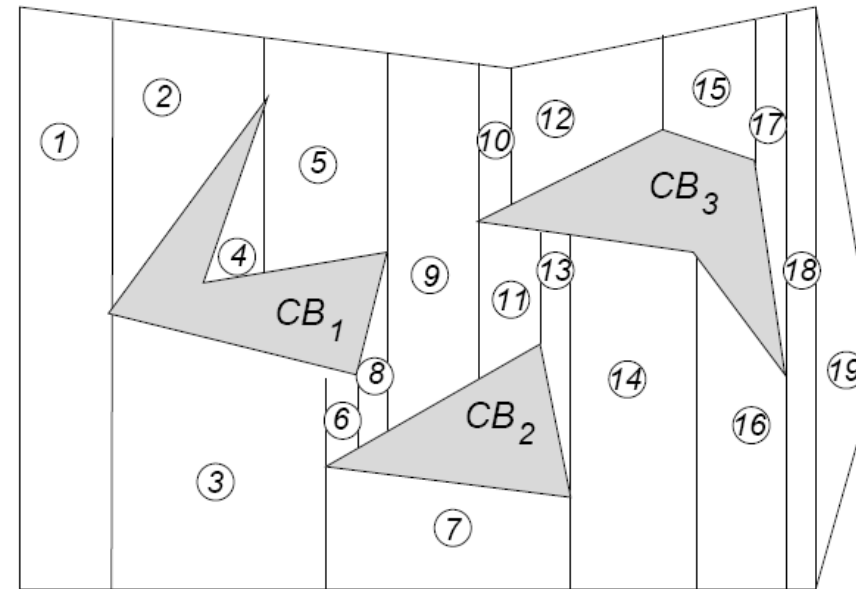


Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact

Types of Cell Decomposition: Trapezoidal

- Complete? **Yes**
- Optimal? **No**
- Advantage? **Speed**



Events: a b c d e f g h i i k l m n o



Defining futures



Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- **Idea:** Construct a collection K of non-overlapping cells such that the union of all the cells approximately (and conservatively) covers the free C-space
- **Cell Characteristics:**
 - Cells should have a standard, simple shape so that it is both easy to compute the decomposition (different from exact decomposition) and easy to compute a path between any two configurations in a cell (same as before)
 - it should be pretty easy to test the adjacency of two cells, i.e., whether they share a boundary (same as before)
 - it should be pretty easy to find a path crossing the portion of the boundary shared by two adjacent cells (same as before)



Defining futures



Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- *Advantages:*
 - simple, uniform decompositions
 - easier and more efficient to implement
 - adaptive—can achieve desired accuracy (resolution)
- *Disadvantages:*
 - large storage requirements
 - lose accuracy and contact info
 - lose completeness
 - decompositions not based on obstacle space

Bottom Line: we sacrifice exactness for simplicity and efficiency



Defining futures

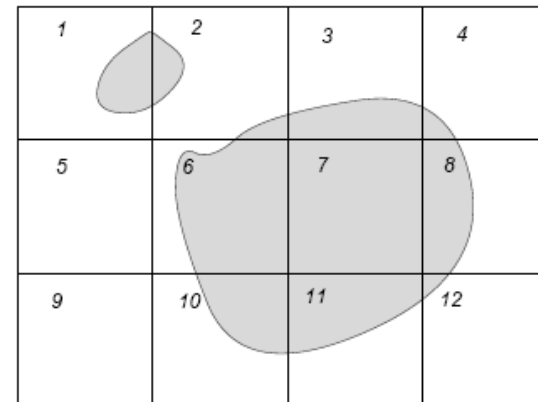


Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- **Decomposition into Rectangloids** : Let Ω be a bounded, closed rectangloid of $\mathcal{R}^m$ representing all configurations of A (m is the dimension of C)
- A **rectangloid** decomposition, P , of Ω is a finite collection of interior disjoint **rectangloids** (cells) $k_1, k_2, \dots, k_r$ such that $\Omega = \bigcup_i k_i$
 - cells k_i and k_j are adjacent if they share a boundary
- Each cell k_i is either:
 - **Empty**: if its interior is contained in C_{free} , i.e., $int(k_i) \cap CB = \emptyset$;
 - **Full**: if it is contained in CB , i.e., $int(k_i) \subseteq CB$
 - **Mixed**: otherwise (its interior is intersected by the boundary of CB)



Defining futures

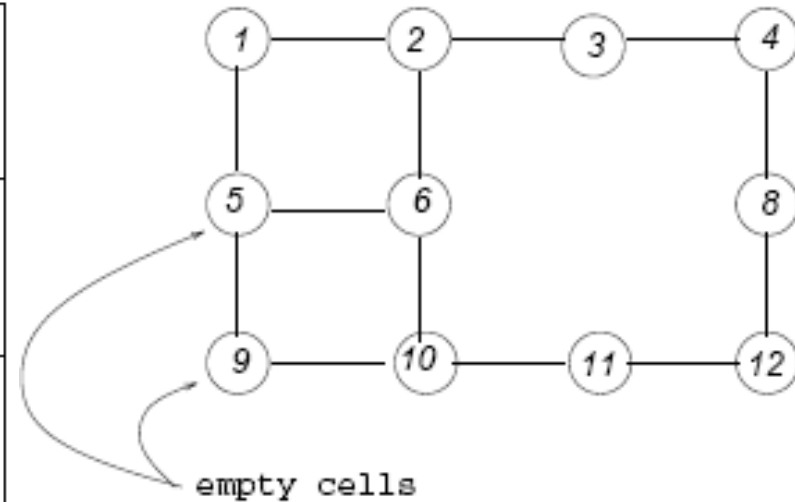
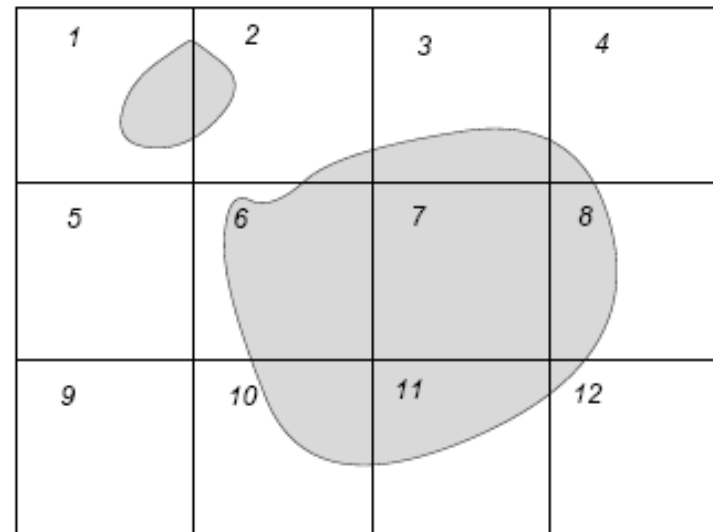


Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- The connectivity graph associated with the decomposition P of is the undirected graph G
 - nodes in G correspond to *Empty* and *Mixed* cells in P
 - nodes connected by edge in G *iff* corresponding cells adjacent in P



Defining futures

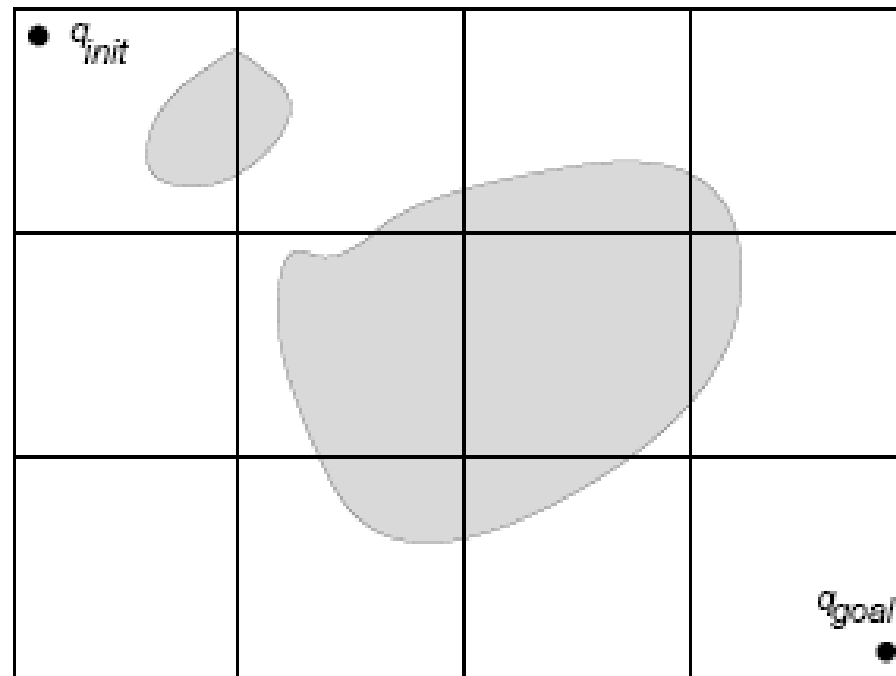


Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- *Channels and Paths in Rectangloid Decompositions* : A channel is a sequence of adjacent *Empty* and *Mixed* cells in P
 - an *E-channel* contains only *Empty* cells
 - an *M-channel* contains at least one *Mixed* cell



Cell Decomposition

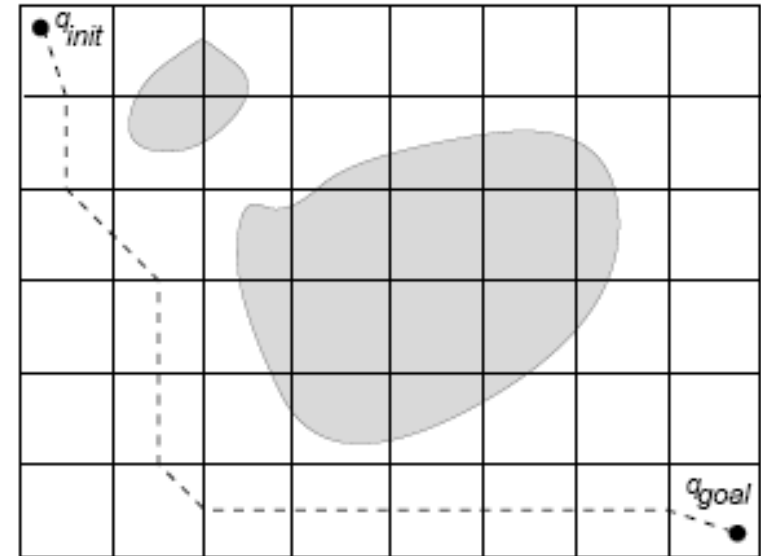


- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- *Path Planning with a Rectangloid Decomposition*

- *input*: configurations q_{init} and q_{goal} , and a rectangloid decomposition P
- *output*: a path in C_{free} connecting q_{init} and q_{goal} ,
 - locate the cells k_{init} and k_{goal} , in P containing q_{init} and q_{goal} ,
 - find an *E-channel* in P between k_{init} and k_{goal} ,
 - find a free path from q_{init} and q_{goal} , in the *E-channel*
- + use midpoints of cell boundaries as crossing points
- + may need to add intermediate interior points if two consecutive mid-points on same of an empty cell



Defining futures

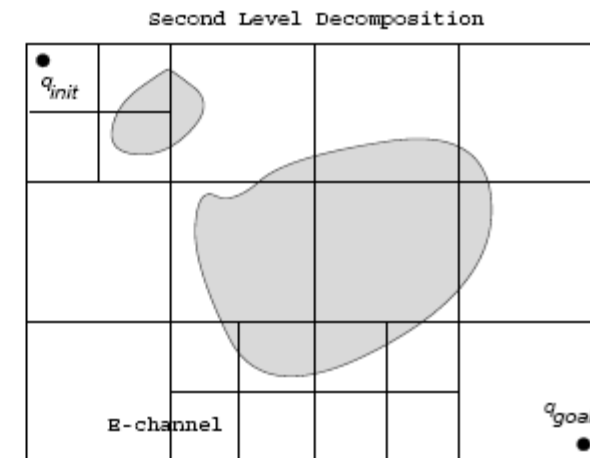
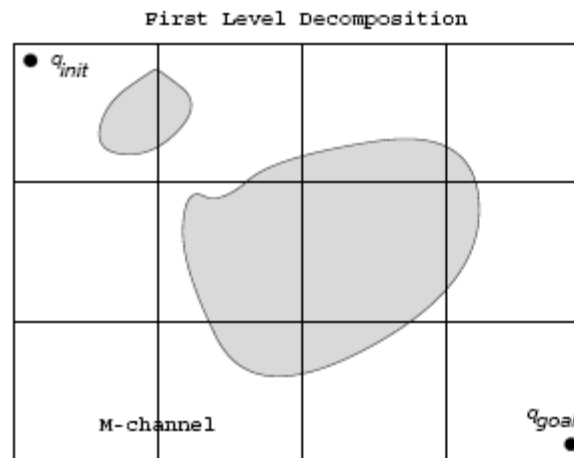


Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- *Hierarchical Path Planning*
 - **Problem**: maybe q_{init} and q_{goal} can only be connected by a ***M-channel*** in P
 - **Idea**: Construct an ***E-channel*** from the ***M-channel*** by constructing rectangloid decompositions of the ***mixed*** cells in the ***M-channel***
 - do this recursively until an ***E-channel*** is obtained (or until no ***M-channel*** exists – no free path exists in channel)
 - let P_i denote the ***i*th** decomposition ($P = P_0$)



Defining futures

Cell Decomposition

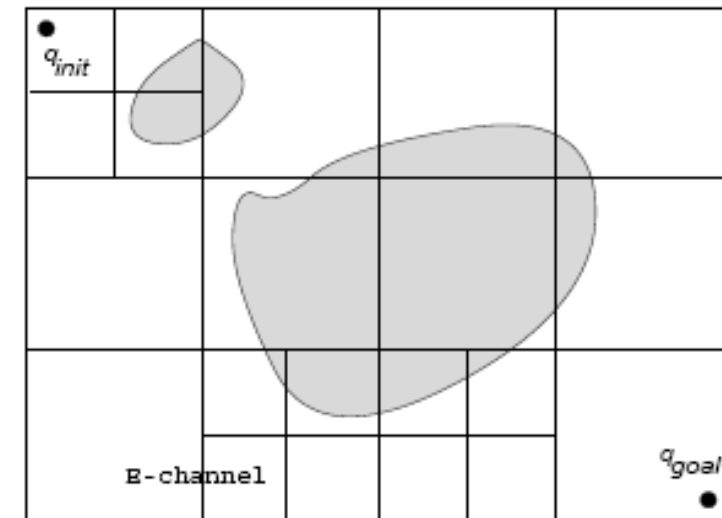


- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- *Hierarchical Path Planning*

- This approach can be made complete, i.e., it will return an *E-channel* whenever q_{init} and q_{goal} lie in the same connected component of C_{free}
 - however, worst-case computation time is *unbounded!*
- Typically, the recursive decomposition is stopped when the size of the cells reaches some fixed value ϵ (resolution)
 - + worst-case computation time is now **bounded** (and adjustable)
 - + method is no longer complete, but it is resolution-complete, i.e., it finds an **E-channel** whenever one exists provided that ϵ is small enough
 - + might stop decomposition if cell is *mostly* full (conservative)



Defining futures



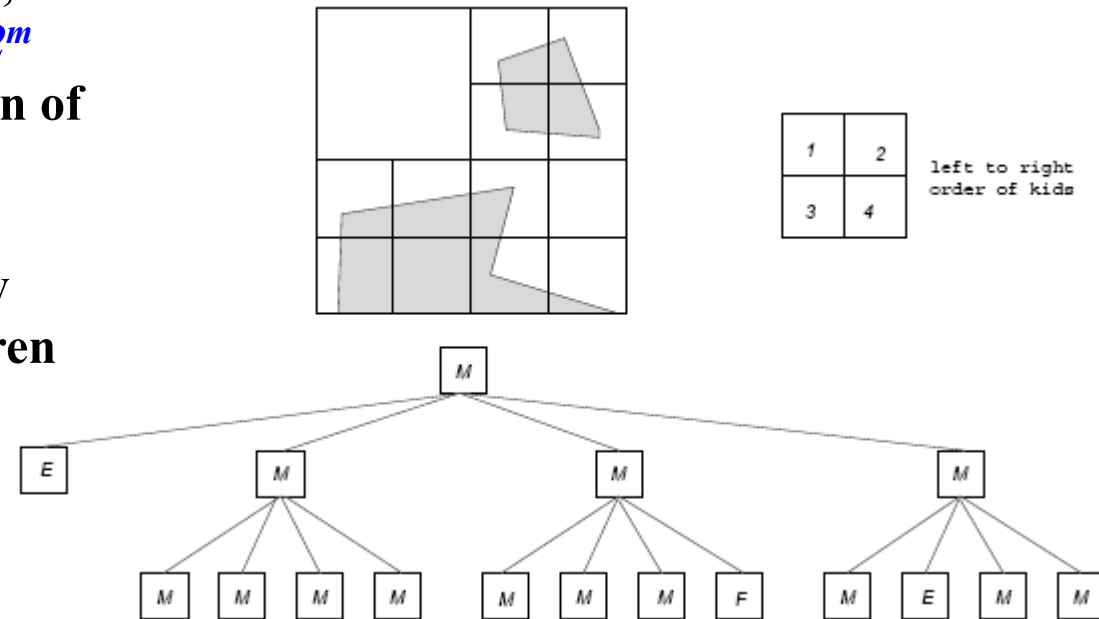
Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- *Decomposing a Mixed Cell: Divide-And-Label*

- A 2^m -Tree Decomposition of *mixed* cell is tree T (m dimension of C):
 - each node is a rectangloid and is **labeled** as *Empty*, *Full*, or *Mixed*
 - the degree of T is 2^m , i.e., each internal node has 2^m children, and all children of a node have the same dimension
 - the root of T is and only *Mixed* nodes have children



Defining futures



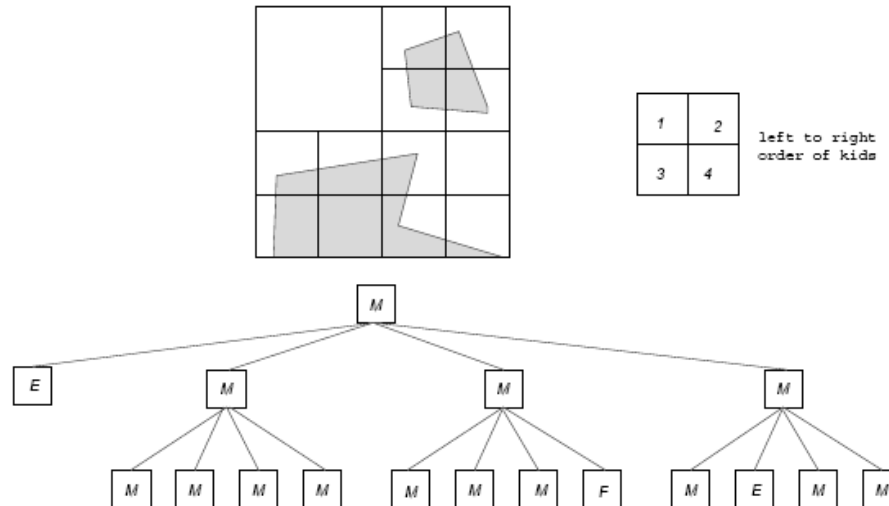
Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate

Types of Cell Decomposition: Approximate

- *Decomposing a Mixed Cell: Divide-And-Label*

- If $m = 2$ we have a *quadtree* and if $m = 3$ we have an *octree*
 - the *depth* of a node (distance from root) determines the dimensions of the cell (depends on size of)
 - the *height* h of the *tree* (*depth* of the deepest *leaf*) determines the resolution of the decomposition (the *deepest node* is the *smallest*)
 - in the worst-case, the *tree* has $2mh$ nodes (in practice usually much less)



Defining futures

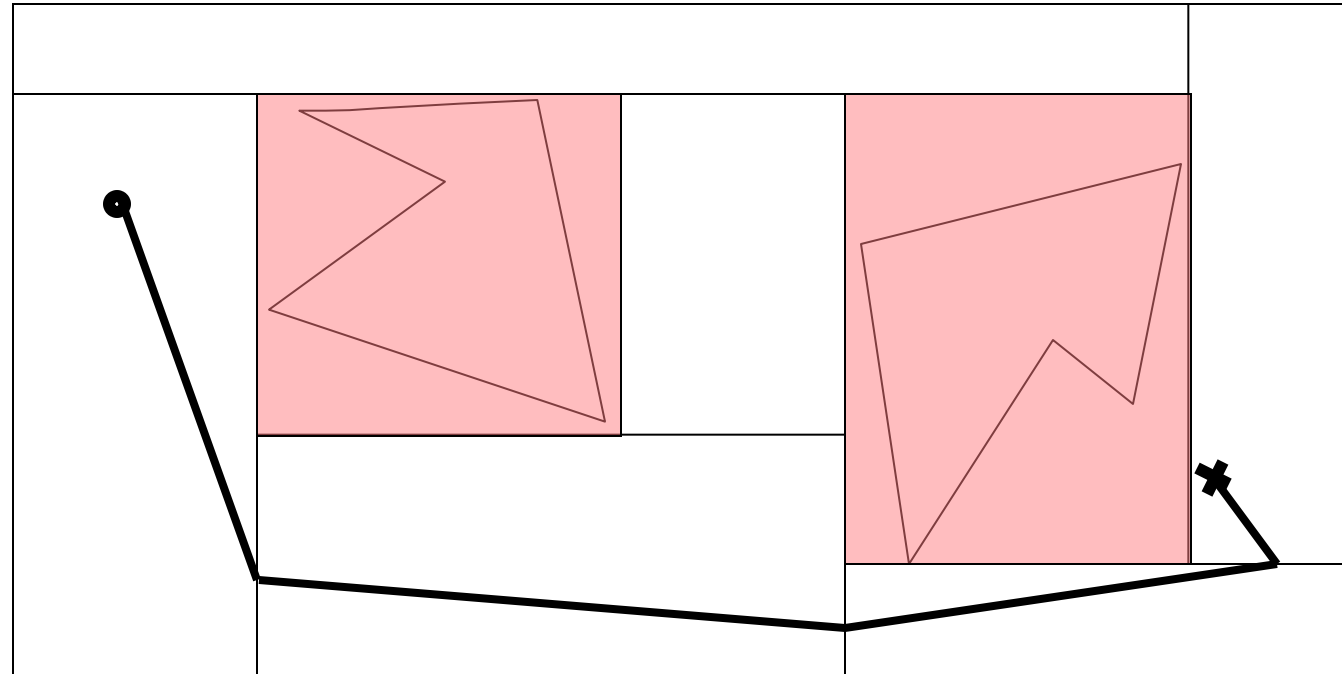


Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate
 - Types

Types of Cell Decomposition: Approximate

- Divide space into (mostly) rectangular regions



Defining futures

Cell Decomposition



- **Cell Decomposition**
 - Basic Idea
 - **Types**
 - Exact
 - Approximate
 - **Types**

Common Types of Approximate Cell Decomposition

- **Common approximate decompositions:**
 - **Regular grids**
 - **Binary or quad trees**
 - **Bounding rectangles**



Defining futures

Cell Decomposition

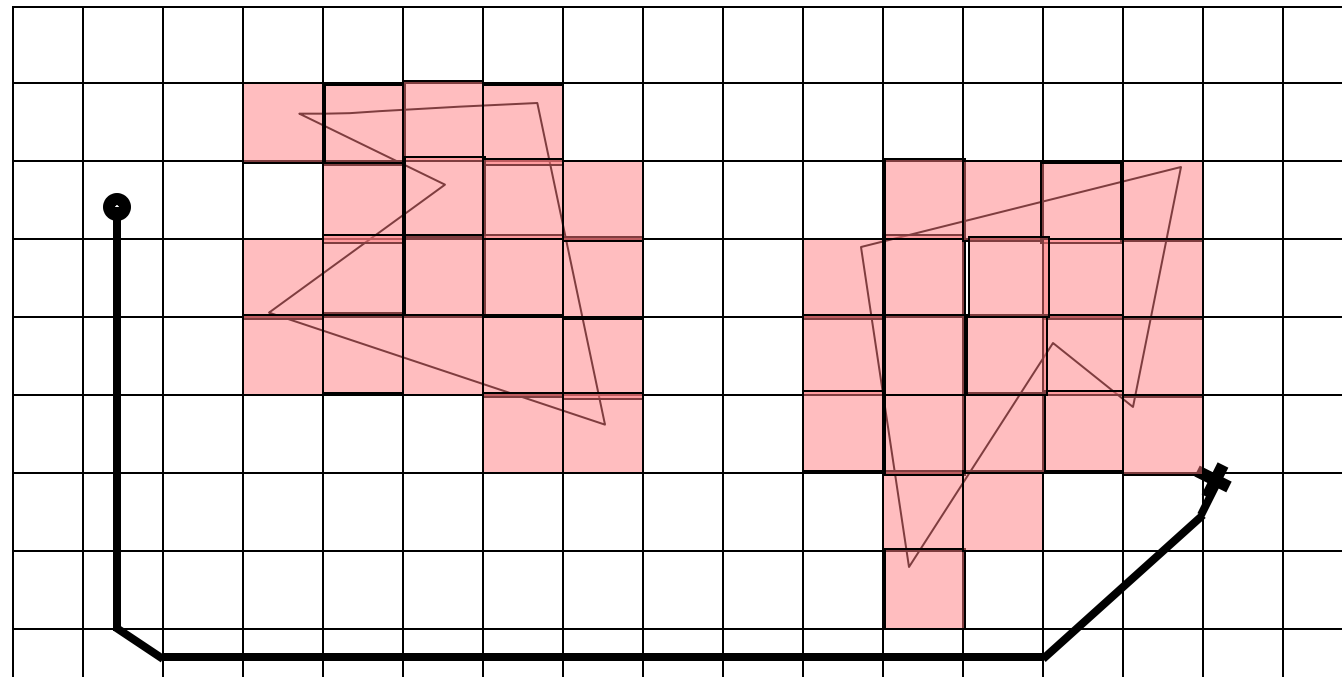


- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate
 - Types
 - **Rectangular**

Common Types of Approximate Cell Decomposition

- *Rectangular Grid Decomposition*

Divide space into a regular grid and mark cells that contain any piece of an obstacle



Defining futures

Cell Decomposition



- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate
 - Types
 - Rectangular

Common Types of Approximate Cell Decomposition

- *Rectangular Grid Decomposition*
 - *Advantages:*
 - Very easy to construct
 - Easy to find cell sequence as each has the same number of neighbors
 - Easy to construct path since edges between cells are predefined
 - *Disadvantages:*
 - Very large number of cells



Defining futures

Cell Decomposition

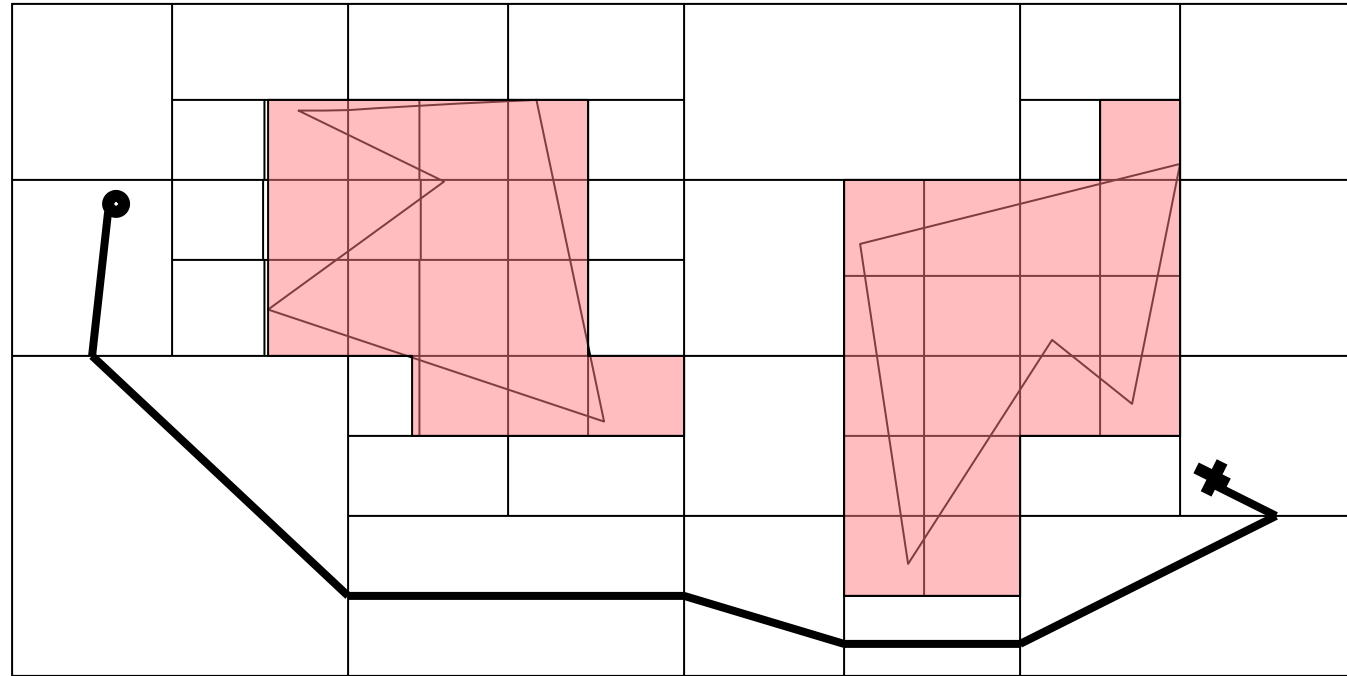


- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate
 - Types
 - Rectangular
 - **Binary Tree**

Common Types of Approximate Cell Decomposition

- *Binary Tree Decomposition*

Divide space by iteratively halving mixed cells vertically or horizontally until a size limit



Defining futures

Cell Decomposition



- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate
 - Types
 - Rectangular
 - **Binary Tree**

Common Types of Approximate Cell Decomposition

- *Binary Tree Decomposition*
 - *Advantages:*
 - Easy to construct
 - Smaller number of cells compared to regular grid at same resolution
 - *Disadvantages:*
 - Higher construction effort as for regular grid



Defining futures

Cell Decomposition

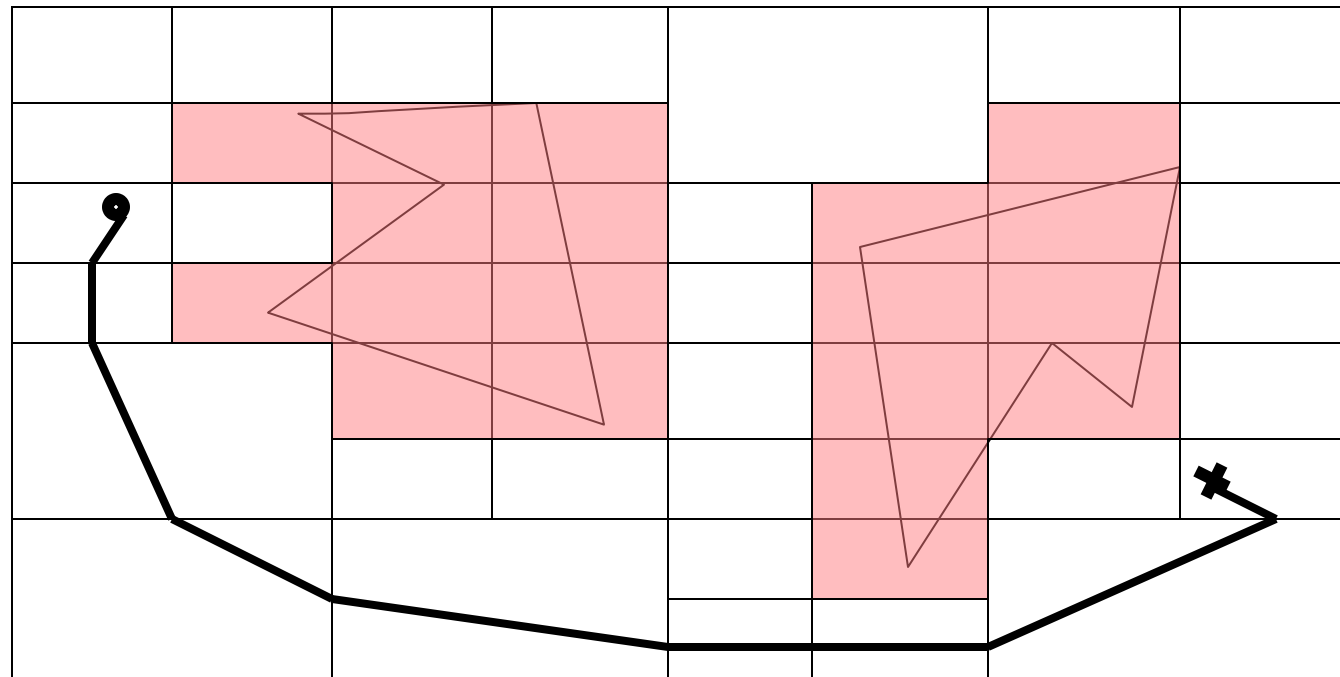


- Cell Decomposition
 - Basic Idea
 - Types
 - Exact
 - Approximate
 - Types
 - Rectangular
 - Binary Tree
 - Quadtree

Common Types of Approximate Cell Decomposition

- *Quad Tree Decomposition*

Divide space by iteratively cutting mixed cells into 4 pieces until a size limit



Defining futures



Cell Decomposition

- Cell Decomposition
 - Basic Idea
 - **Types**
 - Exact
 - Approximate
 - Types
 - Rectangular
 - Binary Tree
 - Quadtree

Common Types of Approximate Cell Decomposition

- *Quad Tree Decomposition*

- *Advantages:*

- Easy to construct
 - Slightly faster than binary decomposition

- *Disadvantages:*

- More cells than binary decomposition at same resolution



Defining futures



Cell Decomposition

- Cell Decomposition

- Basic Idea

- Types

- Exact

- Approximate

- Types

- Rectangular

- Binary Tree

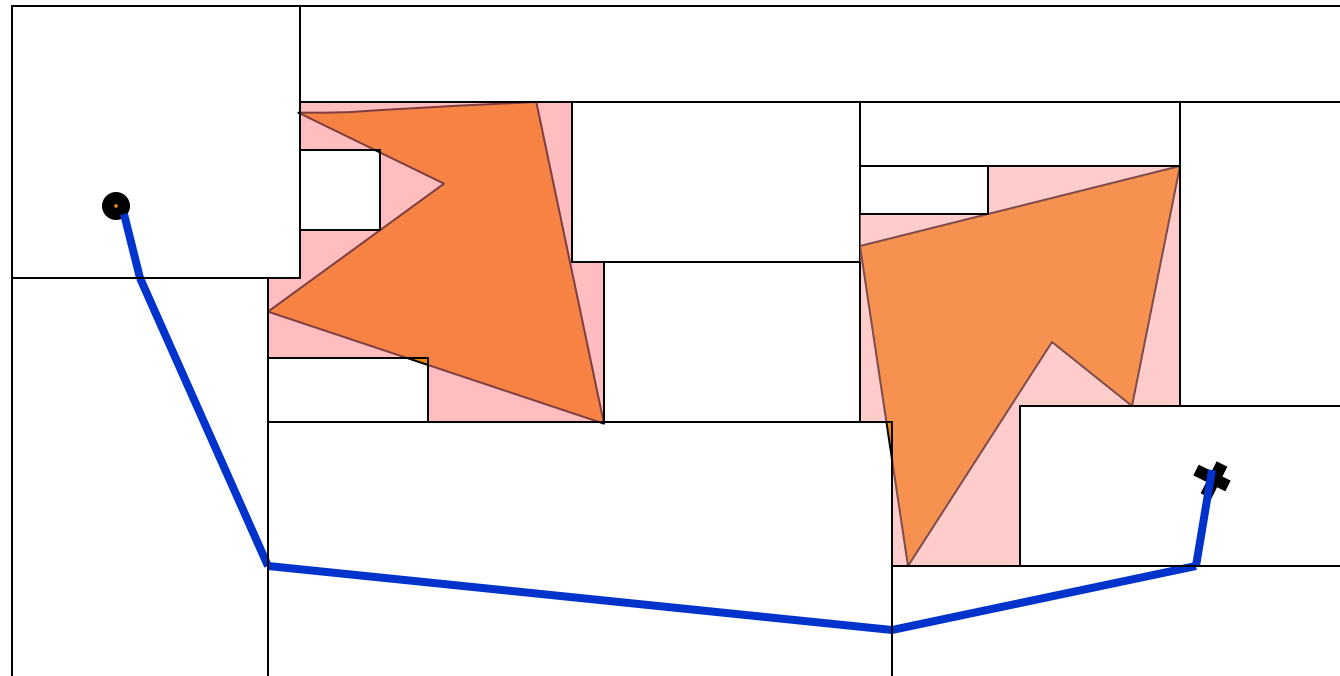
- Quadtree

- Bounding Box

Common Types of Approximate Cell Decomposition

- *Bounding Rectangles*

Enclose free space in bounded rectangles until mixture percentage limit.



Defining futures

Cell Decomposition



- Cell Decomposition

- Basic Idea
- **Types**
 - Exact
 - Approximate
 - Types
 - Rectangular
 - Binary Tree
 - Quadtree
 - **Bounding Box**

Common Types of Approximate Cell Decomposition

- *Bounding Rectangles*

- *Advantages:*
 - Very small number of cells
 - Better coverage than other rectangle-based approaches
- *Disadvantages:*
 - Harder to construct automatically



Defining futures



Cell Decomposition

- Cell Decomposition

- Basic Idea

- **Types**

- Exact

- Approximate

- Types

- Rectangular

- Binary Tree

- Quadtree

- Bounding Box

- Summary

Common Types of Approximate Cell Decomposition

- *Advantages:*

- Faster to construct than exact decomposition
- Easier to represent in data structures since cells are rectangular

- *Disadvantages:*

- Complete only to the resolution of the cells: If no path with sufficient clearance (at least two times the diagonal of the cells) exists, no path might be found.



Defining futures

Cell Decomposition



- **Cell Decomposition**

- Basic Idea
- **Types**
 - Exact
 - Approximate
 - Types
 - Rectangular
 - Binary Tree
 - Quadtree
 - Bounding Box
 - **Summary**

Common Types of Approximate Cell Decomposition

- *Advantages:*

- Often faster to compute than roadmap-based techniques
- Easier to expand into higher dimensional configuration space than roadmap approaches
- Cells are easier to represent than roads

- *Disadvantages:*

- Exact decomposition can be complex
- Approximate decomposition is not complete



Defining futures